Can LLMs Generate High-Quality Test Cases for Algorithm Problems? TestCase-Eval: A Systematic Evaluation of Fault Coverage and Exposure

Anonymous ACL submission

Abstract

We introduce TestCase-Eval, a new benchmark for systematic evaluation of LLMs in test-case generation. TestCase-Eval includes 500 algorithm problems and 10,000 human-crafted solutions from the Codeforces platform. It focuses on two pivotal tasks: (1) *Fault Coverage*, which measures how well LLM-generated test sets probe diverse input scenarios and cover a wide range of potential failure modes. (2) *Fault Exposure*, which evaluates whether LLMs can craft a tailored test input that reveals a specific incorrect code implementation. We provide a comprehensive assessment of 19 state-of-the-art open-source and proprietary LLMs on TestCase-Eval, offering insights into their strengths and limitations in generating effective test cases for algorithm problems.

1 Introduction

Algorithmic problem-solving is fundamental to computational fields such as software engineering, data science, and competitive programming (Jimenez et al., 2023; Huang et al., 2023; Jain et al., 2024; El-Kishky et al., 2025). The correctness and robustness of algorithmic solutions hinge on the quality of test suites—carefully designed inputs that uncover edge cases, corner conditions, performance limitations, and common failure scenarios (Austin et al., 2021; Hendrycks et al., 2021; Li et al., 2022). Traditionally, crafting such test cases requires significant domain expertise and manual effort. However, with the rapid advancement of LLMs capable of sophisticated code generation, a crucial question arises: Can LLMs autonomously generate high-quality test cases that match or surpass those designed by human experts?

We introduce TestCase-Eval[1], a comprehensive benchmark for systematically evaluating LLMs in test-case generation for algorithmic problems. It comprises 500 up-to-date algorithm problems and 10,000 corresponding real-human crafted solutions, both sourced from the Codeforces platform. As illustrated in Figure 1, TestCase-Eval features two core tasks, each targeting a crucial aspect of test-case quality: (1) *Fault Coverage*, which evaluates whether LLM-generated test cases effectively explore diverse input scenarios, including edge cases and boundary conditions, to expose various types of incorrect solutions. (2) *Fault Exposure*, which evaluates whether an LLM can generate a targeted test input that successfully exposes the flaws in a specific given incorrect solution.

We conduct an extensive evaluation on TestCase-Eval, covering 19 frontier open-source and proprietary LLMs. Our experimental results demonstrate that TestCase-Eval presents a significant challenge, with even top-performing models like GPT-4o and Claude-3.5 scoring only around 30% on the Fault Exposure task—far below human expert performance (93.3%). These findings underscore the inherent difficulty of TestCase-Eval. Furthermore, our in-depth analysis of CoT reasoning, code-specific LLMs, and model performance across various programming languages offers valuable insights for future advancements in the field.

2 Related Work

Prior works on LLM-based test-case generation follows two main directions: (1) Enhancing code generation via self-debugging, where models iteratively refine solutions by generating and analyzing test cases (Chen et al., 2022; Zhang et al., 2023; Shinn et al., 2023; Jiao et al., 2024; Zeng et al., 2025). (2) Improving code evaluation by leveraging LLMs to generate diverse test cases, as used in recent code generation evaluation benchmarks (Liu et al., 2023; Du et al., 2024; Yu et al., 2024; Jain et al., 2024). Despite these advancements, a sys-

[1]The data and code can be found at https://anonymous.4open.science/r/TestCase-Eval, and will be released publicly upon publication.

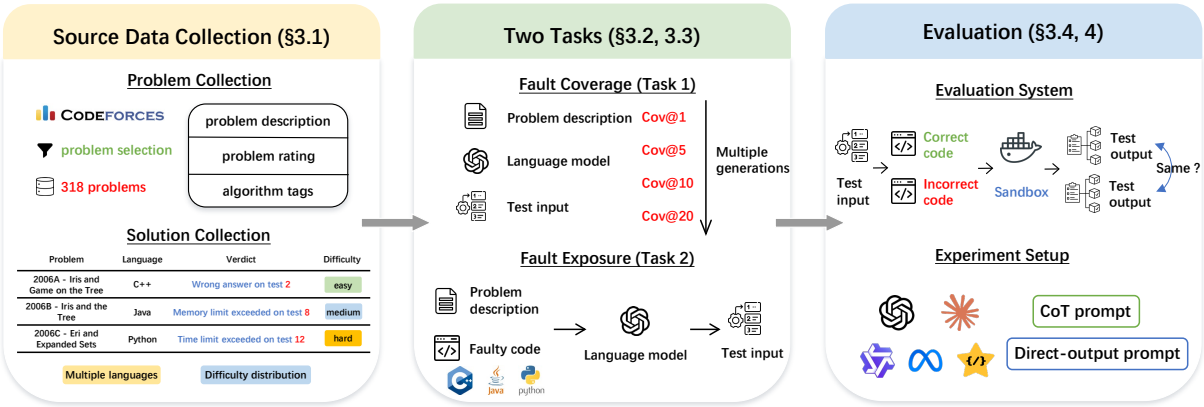


Figure 1: An overview of TestCase-Eval and the research pipeline in this study.

tematic study on the standalone capability of LLMs in test-case generation remains an open challenge. A closely related benchmark for algorithmic problem test-case generation is TestEval (Wang et al., 2024), which evaluates test generation for LeetCode problems but relies on traditional Line and Branch Coverage assessment (illustrated in Appendix A.1), which may be inadequate for algorithmic problem settings. For instance, the 6.7B DeepSeek-Coder achieves over 90% in TestEval, with top models nearing 100%. Our work shifts the focus to more challenging CodeForces competition problems and introduces two novel and challenging tasks. Even state-of-the-art models achieve only around 30% on the task of Fault Exposure.

3 TestCase-Eval Benchmark

This section discusses the TestCase-Eval task settings, benchmark construction, and automated evaluation system, as illustrated in Figure 1.

3.1 Source Data Collection

We first outline the data collection process, which involves collecting *problems* with corresponding correct and incorrect *human-written solutions*.

Problem Collection. We collect algorithmic problems from Codeforces contests held between January 1, 2024, and December 30, 2024. This time frame falls outside the pretraining period of most existing foundation models, reducing potential data memorization concerns. Our goal is to curate a dataset that (1) includes a substantial number of incorrect submissions and (2) ensures accurate offline evaluation. To achieve this, we apply a series of filtering steps. First, we exclude problems requiring special judge functionalities (detailed in Appendix B.3), as these allow multiple valid outputs for a single input, often leading to unreliable evaluations. Next, we verify each problem by running ten correct human-written solutions sourced from Codeforces, ensuring they consistently produce identical outputs for the same test inputs. Additionally, we remove problems with fewer than 1,000 online incorrect submissions to ensure a diverse range of mistakes. After this selection process, our final dataset includes a total of 500 problems. Figure 3 in Appendix presents the distribution of problem difficulty ratings.

Human-written Solution Collection. For each problem in TestCase-Eval, we collect 200 incorrect submissions (*i.e.,* human-written solutions along with their evaluation outcomes) from the Codeforces platform. The platform provides detailed error types for each incorrect submission, such as "Memory Limit Exceeded", "Time Limit Exceeded", "Runtime Error", and "Wrong Answer". Additionally, it specifies the test case index where the error occurred (*e.g.,* "Wrong answer on test 5"), with higher indices generally indicating more complex or inherent errors. Leveraging this information, we categorize incorrect submissions into three difficulty levels: *Easy*, *Medium*, and *Hard*. (Detailed difficulty definition and distribution are illustrated in Appendix B.2) To ensure diversity, we select code written in three widely used programming languages: C++, Python, and Java. We begin by crawling the submission logs for each problem, which contain the complete history of contestant submissions. These logs are carefully filtered based on test case failures and programming language criteria, resulting in a comprehensive dataset of 10,000 submissions.

We next outline the construction process for the

two evaluation tasks in TestCase-Eval.

3.2 Fault Coverage Evaluation (Task 1)

This task assesses the LLM's ability to generate comprehensive test inputs that effectively detect faulty code implementations. Specifically, given the description of an algorithmic problem, the LLM must thoroughly understand the problem and generate a specified number of test cases that maximize coverage of incorrect solution scenarios. Let $\mathcal{T}_N = \{t_1, t_2, \ldots, t_N\}$ represent the set of N test inputs generated by the LLM. For each test input t_i, let $\mathcal{F}(t_i)$ denote the subset of incorrect submissions it detects, drawn from the complete set of incorrect solutions $\mathcal{F}_{\text{total}}$, the final score for this task is defined as the coverage rate of incorrect solutions when generating N test inputs:

$$Cov@N = \frac{\left|\bigcup_{i=1}^{N} \mathcal{F}(t_i)\right|}{|\mathcal{F}_{\text{total}}|}$$

It quantifies the LLM's effectiveness in generating diverse and impactful test cases that expose incorrect implementations.

3.3 Fault Exposure Evaluation (Task 2)

This task is inspired by the hacking phase in CodeForces competitions, where participants analyze others' solutions and attempt to "hack" them by providing inputs that reveal flaws in the code. The goal is to assess the LLM's ability to understand both the problem and the specific errors present in the faulty implementation. Given the description of an algorithmic problem and a single faulty code implementation f_i within the set $\mathcal{F}$, Task 2 requires the LLM to generate a single test input t_i to exploit the fault. The *Fault Exposure Rate* is computed as:

$$\text{Fault Exposure rate} = \frac{1}{|\mathcal{F}|} \sum_{f_i \in \mathcal{F}} e(f, t_f), \text{ where}$$

$$e(f_i, t_i) = \begin{cases} 1, & \text{if } t_i \text{ successfully exposes fault in } f_i, \\ 0, & \text{otherwise.} \end{cases}$$

It measures both general test case generation capabilities and targeted fault detection performance.

4 Experiment

4.1 Experiment Setup

We evaluate 11 series of open-source models, including Qwen2&2.5 and Qwen2.5-Coder (Hui et al., 2024), QwQ (Team, 2024b), Llama-3.1&3.3 (Meta, 2024), Gemma-2 (Google, 2024), StarCoder2 and StarChat2 (Lozhkov et al., 2024), DeepSeek-R1 (DeepSeek-AI et al., 2025), Mistral and Codestral (Team, 2024a). We also evaluate two series of proprietary models, including GPT-4o (OpenAI, 2024) and Claude-3.5 (Anthropic, 2024). Appendix C.1 details the parameter settings and model configurations. We evaluate the models with both **Direct Output** and **Chain-of-Thought** prompts, with prompting examples presented in Appendix C.2. We utilize the **sandbox environment** from ExecEval (Khan et al., 2023) for code execution and test input evaluation, ensuring secure execution and reliable assessment of results. To approximate **human-expert-level performance** on TestCase-Eval, we randomly sampled 20 problems from the dataset. Two human experts, with Codeforces ratings of 2080 and 2237, independently completed both Task 1 and Task 2 for each problem. Their performance was then averaged to obtain the final assessment.

4.2 Experimental Results and Analysis

Table 1 illustrates the model performance on TestCase-Eval. Our key findings are as follows:

TestCase-Eval presents substantial challenges for current models. The TestCase-Eval benchmark is highly challenging, as evidenced by the consistently lower performance of models on both tasks. In contrast, human experts significantly outperform all models, achieving over 90% on the Fault Exposure Task, whereas the best-performing model (Claude-3.5-Sonnet) reaches only 32.5%.

Proprietary Models outperform open-source models. Proprietary models, such as GPT-4o and Claude-3.5, tend to outperform their open-source counterparts, consistently achieving higher fault coverage and better fault exposure. For example, GPT-4o exhibits higher coverage in Task 1 (Fault Coverage) at Cov@1 (31.6), Cov@5 (42.8), Cov@10 (49.0), and Cov@20 (51.6) compared to models like Llama-3.1-70B and Qwen2.5-Coder-32B. Additionally, in terms of fault exposure, proprietary models significantly surpass the open-source models, demonstrating that advanced proprietary models may be better optimized for the type of reasoning required in the TestCase-Eval tasks.

Code LLMs do not consistently outperform their base models on TestCase-Eval. Despite

Models	T1: Fault Coverage				T2: Fault Exposure			
	c@1	c@5	c@10	c@20	Easy	Med.	Hard	**Ovr.**
Human Expert	37.5	57.5	70.0	85.0	95.0	92.5	92.5	93.3
Claude-3-5-Sonnet	26.4	37.4	45.1	50.1	42.3	29.3	22.0	32.5
GPT-4o	31.6	42.8	49.0	51.6	41.4	28.1	21.9	31.6
GPT-4o-mini	31.6	42.1	48.4	52.0	25.7	22.8	17.5	22.4
Llama-3.1-70B	22.0	29.4	33.3	45.5	28.4	26.3	21.7	25.8
Qwen2.5-Coder-32B	23.9	33.1	42.5	49.7	30.6	26.7	17.3	25.4
Llama-3.1-8B	19.7	29.4	36.8	42.5	26.8	22.0	20.2	23.4
Qwen2.5-32B	32.6	40.3	47.1	57.2	29.2	23.7	14.0	23.0
QwQ-32B-Preview	23.2	36.6	41.8	48.4	28.3	20.0	15.6	22.0
DS-R1-Distill-32B	19.7	33.6	40.1	43.6	29.2	18.9	15.0	21.8
Qwen2.5-Coder-7B	24.0	34.8	44.7	56.1	24.6	22.2	16.3	21.4
Mistral-Small-24B	16.6	27.8	33.3	44.1	24.8	22.3	15.0	21.1
Qwen2.5-72B	20.0	28.1	30.4	36.1	24.6	21.5	13.5	20.3
Llama-3.3-70B	29.9	32.8	37.0	44.4	26.2	19.0	13.3	20.2
Gemma-2-9B	12.5	26.1	34.0	42.1	22.2	20.7	17.2	20.2
Gemma-2-27B	13.7	24.4	34.6	42.5	24.5	17.7	14.5	19.4
Qwen2-7B	15.8	24.8	38.9	45.9	22.0	18.3	15.2	18.9
Codestral-22B	22.6	31.1	40.0	48.8	21.5	17.3	13.5	17.8
StarChat2-15B	17.9	26.2	34.9	51.3	18.9	18.7	14.2	17.4
StarCoder2-15B	8.1	13.7	19.2	26.8	11.0	10.5	8.2	10.0

Table 1: Performance of the evaluated LLMs with CoT reasoning on TestCase-Eval. For Task 1, we report Coverage@N; for Task 2, we report fault exposure rate.

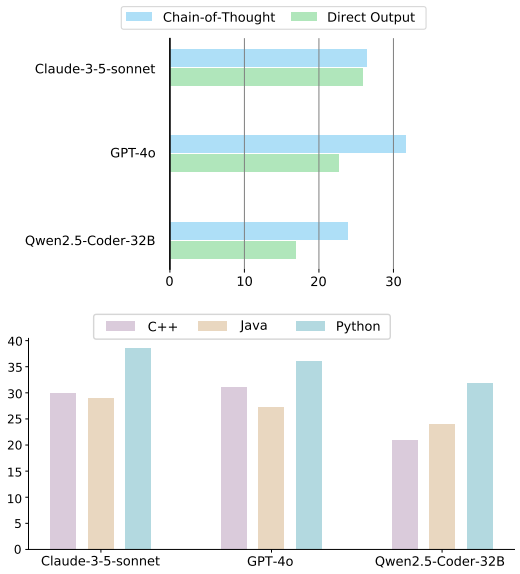


Figure 2: (Top) Comparison of model performance in Task 1: Evaluating cov@1 metrics between CoT prompting and direct output prompting. (Bottom) Assessment of overall model performance using CoT prompting across C++, Python, and Java in Task 2.

being designed specifically for code generation tasks, Code LLMs do not consistently surpass their general-purpose counterparts on TestCase-Eval. For example, while Qwen2.5-Coder-32B is fine-tuned for coding, it does not significantly outperform its base model (*i.e.,*, Qwen2.5-32B) in generating test cases. This suggests that task-specific tuning does not always yield the expected benefits, particularly for benchmarks like TestCase-Eval, which require a nuanced understanding of algorithmic problems and error detection across multiple programming languages.

CoT prompts vs Direct-output prompts. Our experiments reveal that CoT prompting significantly outperforms direct-output prompting in generating test cases (Figure 2). This advantage stems from CoT's structured reasoning process, which guides the model through intermediate steps before arriving at the final output. Such an approach is particularly beneficial for the complex tasks in TestCase-Eval, where systematic thinking is crucial. When generating test cases for fault coverage, CoT prompting led to more effective fault detection, especially in challenging edge cases. This suggests that models benefit from explicit reasoning steps, as they help decompose intricate problems and improve coverage in nuanced scenarios.

Comparison of Fault Exposure results across different programming languages. In Fault Exposure task, model performance varies across programming languages. As shown in Figure 2, models generally achieve higher fault exposure rates on Python solutions, likely due to Python's dynamic typing, flexible syntax, and interpreted execution, which facilitate the generation of diverse test cases that reveal faults. In contrast, C++ and Java exhibit lower fault exposure rates, possibly due to their strict type enforcement, manual memory management, and compiled execution, which can limit the likelihood of generating inputs that reveal subtle faults. Addressing these differences through targeted adaptation could help improve fault detection across a wider range of programming languages.

5 Conclusion

We introduce TestCase-Eval, a new benchmark for evaluating LLMs in test-case generation for algorithmic problems, with a focus on Fault Coverage and Fault Exposure. Our results show that all the evaluated LLMs struggle with harder faults, highlighting the challenge of automated test-case generation. Although CoT prompting enhances performance, a substantial gap remains between frontier LLMs and human experts. These findings emphasize the need for further research to enhance LLMs' capabilities in generating high-quality test cases and their practical application in software testing.

Limitations

While TestCase-Eval provides a comprehensive benchmark for evaluating LLMs in test-case generation, there are several limitations that warrant further investigation: (1) Our evaluation primarily emphasizes quantitative performance indicators, such as fault coverage and exposure rates. This focus might not capture the nuanced failure modes that may arise in LLM-generated test cases. Future work could delve into more detailed error analyses to uncover specific failure patterns and model weaknesses. (2) The difficulty levels (Easy, Medium, Hard) in TestCase-Eval are determined based on the test case index where an incorrect solution first fails. While this approach provides a reasonable estimate of error complexity, it does not explicitly categorize the types of mistakes made by incorrect implementations. (3) Our benchmark primarily evaluates correctness-based faults and does not systematically test performance bottlenecks (*e.g.,* time limit exceeded, memory limit exceeded). While some incorrect submissions in our dataset fail due to resource constraints, we do not explicitly assess whether LLMs generate test cases that effectively expose computational complexity flaws. (4) Our experiments evaluate LLMs in an autonomous test-case generation setting, but practical software testing often involves human oversight. The interplay between human engineers and LLM-generated test cases remains an open question.

References

Anthropic. 2024. Introducing the next generation of claude.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. Program synthesis with large language models. *ArXiv*, abs/2108.07732.

Bei Chen, Fengji Zhang, A. Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. Codet: Code generation with generated tests. *ArXiv*, abs/2207.10397.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Jun-Mei Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiaoling Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bing-Li Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dong-Li Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Jiong Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, M. Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaokang Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wen-Xia Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyu Jin, Xi-Cheng Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yi Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yu-Jing Zou, Yujia He, Yunfan Xiong, Yu-Wei Luo, Yu mei You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yao Li, Yi Zheng, Yuchen Zhu, Yunxiang Ma, Ying Tang, Yukun Zha, Yuting Yan, Zehui Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhen guo Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zi-An Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning.

Mingzhe Du, Anh Tuan Luu, Bin Ji, Qian Liu, and See-Kiong Ng. 2024. Mercury: A code efficiency benchmark for code large language models. In *Neural Information Processing Systems*.

OpenAI Ahmed El-Kishky, Alexander Wei, Andre Saraiva, Borys Minaev, Daniel Selsam, David Dohan, Francis Song, Hunter Lightman, Ignasi Clavera, Jakub W. Pachocki, Jerry Tworek, Lorenz Kuhn, Lukasz Kaiser, Mark Chen, Max Schwarzer, Mostafa Rohaninejad, Nat McAleese, o3 contributors, Oleg Murk, Rhythm Garg, Rui Shu, Szymon Sidor, Vineet Kosaraju, and Wenda Zhou. 2025. Competitive programming with large reasoning models.

Google. 2024. Gemma 2: Improving open language models at a practical size. *ArXiv*, abs/2408.00118.

Hugging Face H4. 2024. Model card for starchat2 15b.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Xiaodong Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with apps. *ArXiv*, abs/2105.09938.

Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. 2023. Mlagentbench: Evaluating language agents on machine learning experimentation. In *International Conference on Machine Learning*.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, An Yang, Rui Men, Fei Huang, Shanghaoran Quan, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-coder technical report. *ArXiv*, abs/2409.12186.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida I. Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. *ArXiv*, abs/2403.07974.

Albert Qiaochu Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L'elio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7b. *ArXiv*, abs/2310.06825.

Fangkai Jiao, Geyang Guo, Xingxing Zhang, Nancy F. Chen, Shafiq Joty, and Furu Wei. 2024. Preference optimization for reasoning with pseudo feedback. *ArXiv*, abs/2411.16345.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *ArXiv*, abs/2310.06770.

Mohammad Abdullah Matin Khan, M Saiful Bari, Do Xuan Long, Weishi Wang, Md. Rizwan Parvez, and Shafiq R. Joty. 2023. xcodeeval: A large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval. *ArXiv*, abs/2303.03004.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom, Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de, Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey, Cherepanov, James Molloy, Daniel Jaymin Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de, Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with alphacode. *Science*, 378:1092 – 1097.

Jiawei Liu, Chun Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *ArXiv*, abs/2305.01210.

Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan L. Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, W. Yu, Lucas Krauss, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alexander Gu, Binyuan Hui, Tri Dao, Armel Randy Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian J. McAuley, Han Hu, Torsten Scholak, Sébastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Muñoz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2024. Starcoder 2 and the stack v2: The next generation. *ArXiv*, abs/2402.19173.

Meta. 2024. The llama 3 herd of models. *ArXiv*, abs/2407.21783.

OpenAI. 2024. Hello gpt-4o.

Noah Shinn, Federico Cassano, Beck Labash, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Neural Information Processing Systems*.

Mistral AI Team. 2024a. Codestral: Hello, world!

Qwen Team. 2024b. Qwq: Reflect deeply on the boundaries of the unknown.

Wenhan Wang, Chenyuan Yang, Zhijie Wang, Yuheng Huang, Zhaoyang Chu, Da Song, Lingming Zhang, An Ran Chen, and Lei Ma. 2024. Testeval: Benchmarking large language models for test case generation. *ArXiv*, abs/2406.04531.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Ke-Yang Chen, Kexin Yang, Mei Li, Min Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yunyang Wan, Yunfei Chu, Zeyu Cui, Zhenru Zhang, and Zhi-Wei Fan. 2024a. Qwen2 technical report. *ArXiv*, abs/2407.10671.

Qwen An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxin Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yi-Chao Zhang, Yunyang Wan, Yuqi Liu, Zeyu Cui, Zhenru Zhang, Zihan Qiu, Shanghaoran Quan, and Zekun Wang. 2024b. Qwen2.5 technical report. *ArXiv*, abs/2412.15115.

Zhaojian Yu, Yilun Zhao, Arman Cohan, and Xiao-Ping Zhang. 2024. Humaneval pro and mbpp pro: Evaluating large language models on self-invoking code generation. *ArXiv*, abs/2412.21199.

Huaye Zeng, Dongfu Jiang, Haozhe Wang, Ping Nie, Xiaotong Chen, and Wenhu Chen. 2025. Acecoder: Acing coder rl via automated test-case synthesis.

Kexun Zhang, Danqing Wang, Jingtao Xia, William Yang Wang, and Lei Li. 2023. Algo: Synthesizing algorithmic programs with generated oracle verifiers. *ArXiv*, abs/2305.14591.

A Related Work

A.1 Line and Branch Coverage

In the realm of software testing, two pervasive metrics are Line Coverage and Branch Coverage. These are often used to evaluate the adequacy of test cases in executing program code.

Specifically, **Line Coverage** measures the percentage of lines of code that have been executed by a set of test cases. It provides insight into which lines of the codebase are actually executed during testing, aiming to ensure that all parts of the code are tested at least once. **Branch Coverage** takes a more granular approach by focusing on the control structures within the code, such as if statements and switch cases. It evaluates whether each possible branch (i.e., each path through a control structure) has been executed. This metric ensures that all possible execution paths are tested.

While these metrics are invaluable in traditional software testing, they fall short in the context of competitive algorithm problems for several reasons: (1) Algorithm Complexity and Diversity: Competitive algorithm problems often involve complex data structures and intricate algorithmic logic that cannot be fully represented by simple line or branch execution. The focus is on the correctness and efficiency of the algorithm, rather than merely executing each line or branch of code. (2) Outcome-Oriented Nature: The primary goal in algorithm competitions is to solve problems correctly and efficiently, not just to achieve high code coverage. An algorithm may achieve high line and branch coverage but still fail to solve the problem correctly or efficiently. (3) Diversity of Test Cases: Algorithm competition problems require testing against a wide variety of edge cases and specific inputs. The generation and evaluation of these test cases extend beyond the scope of simple line and branch coverage metrics, which may not adequately reflect the comprehensiveness of the test cases in ensuring algorithmic correctness and robustness. Traditional Line and Branch Coverage metrics may be inadequate in the context of algorithmic problems. Therefore, we propose two new tasks, Fault Coverage and Fault Exposure, along with corresponding evaluation metrics.

B TestCase-Eval Benchmark

We provide a detailed explanation of the problems and human-written solutions within the dataset.

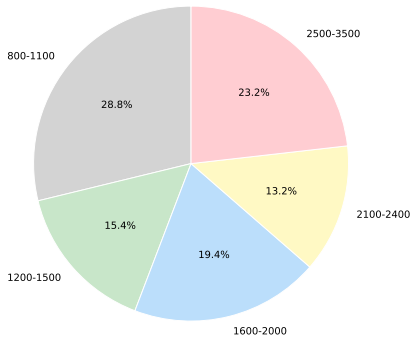


Figure 3: Distribution of Problem Difficulty Levels.

B.1 Problems

Each problem sourced from the Codeforces platform comprises several key elements: 1) title, 2) time limit, 3) memory limit, 4) problem description, 5) input format, 6) output format, 7) test case examples, and 8) optional note. We utilize all of this data to form the `problem_description` string, which acts as the input for the LLM.

Additionally, we analyze the distribution of problem difficulty ratings, which is illustrated in Figure Figure 3.

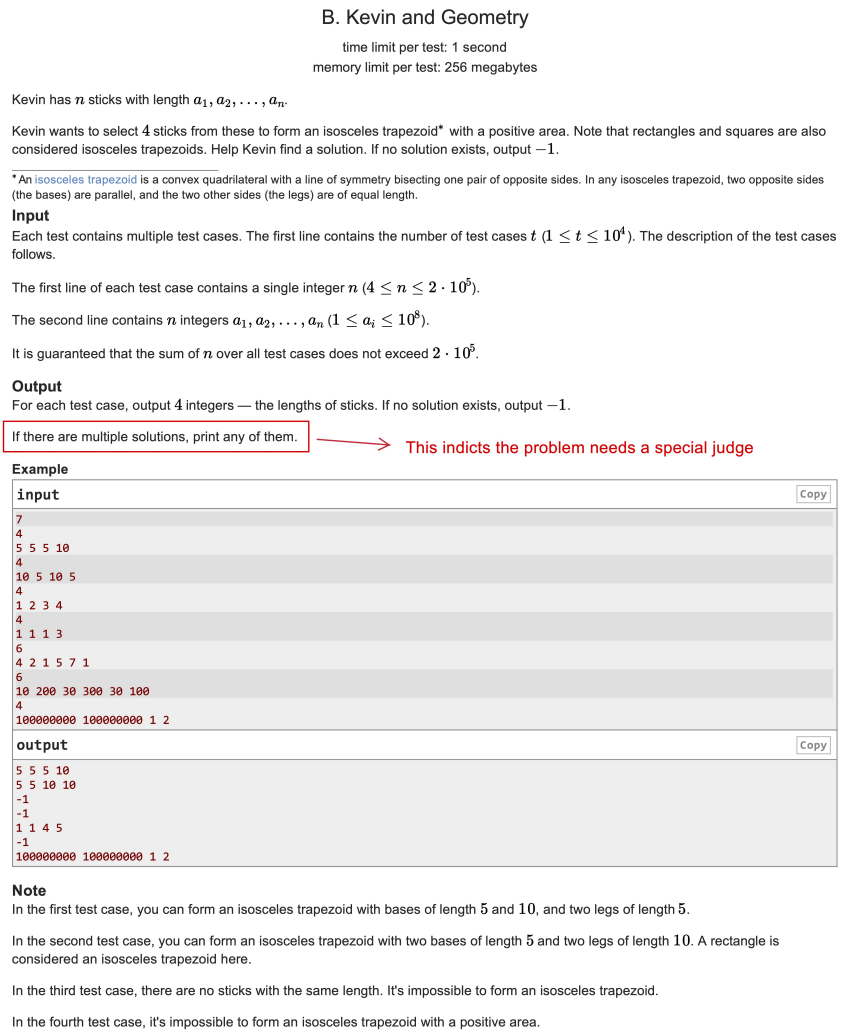
B.2 Human-written Solutions

Typically, Codeforces problems include multiple test cases, which can number from a dozen to over two hundred. Each test case includes a set of inputs and the expected outputs. A verdict such as "Wrong answer on test 5" implies that the solution was correct for the first four test cases but failed on the fifth one.

The index of the test case where the first error appears is crucial for evaluating LLM-generated test cases through incorrect submissions, which serves as the standard for difficulty classification. We classify these solutions into three difficulty levels based on this test case index: *Easy*, *Medium*, and *Hard*. Specifically, we sort all human-written solutions by the test case index of the first error, and classify the lower 40% as *Easy*, the middle 30% as *Medium*, and the top 30% as *Hard*.

B.3 Special Judge

In competitive programming platforms like Codeforces, certain problems permit multiple valid test outputs for a single test input. To validate such outputs, a special judge is employed. This custom code evaluates the correctness of each output, as straightforward comparison to a reference output is inadequate due to the problem's complexity.



B. Kevin and Geometry

time limit per test: 1 second
memory limit per test: 256 megabytes

Kevin has n sticks with length $a_1, a_2, \ldots, a_n$.

Kevin wants to select 4 sticks from these to form an isosceles trapezoid* with a positive area. Note that rectangles and squares are also considered isosceles trapezoids. Help Kevin find a solution. If no solution exists, output -1.

*An isosceles trapezoid is a convex quadrilateral with a line of symmetry bisecting one pair of opposite sides. In any isosceles trapezoid, two opposite sides (the bases) are parallel, and the two other sides (the legs) are of equal length.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 10^4$). The description of the test cases follows.

The first line of each test case contains a single integer n ($4 \le n \le 2 \cdot 10^5$).

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^8$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output 4 integers — the lengths of sticks. If no solution exists, output -1.

If there are multiple solutions, print any of them.

This indicts the problem needs a special judge

Example

input

```
7
4
5 5 5 10
4
10 5 10 5
4
1 2 3 4
4
1 1 1 3
6
4 2 1 5 7 1
6
10 200 30 300 30 100
4
100000000 100000000 1 2
```

output

```
5 5 5 10
5 5 10 10
-1
-1
1 1 4 5
-1
100000000 100000000 1 2
```

Note

In the first test case, you can form an isosceles trapezoid with bases of length 5 and 10, and two legs of length 5.

In the second test case, you can form an isosceles trapezoid with two bases of length 5 and two legs of length 10. A rectangle is considered an isosceles trapezoid here.

In the third test case, there are no sticks with the same length. It's impossible to form an isosceles trapezoid.

In the fourth test case, it's impossible to form an isosceles trapezoid with a positive area.

Figure 4: An example of a problem that needs a special judge.

Figure 4 illustrates a problem that necessitates a special judge.

For accurate offline evaluation, we excluded all problems requiring a special judge. Such problems can cause inconsistent assessments since they allow multiple correct outputs for the same input.

C Experiment Setup

C.1 Evaluated Model Configuration

Table 2 details the configuration of each evaluated model. Across all experiments, the temperature is set to 1.0 to ensure diversity in the LLM-generated test cases. The maximum output length is generally configured to 2048 tokens, which suffices for most standard models. However, for reasoning models like QwQ-32B-Preview and R1-Distill-Qwen-32B, this maximum output length is extended to 8192 tokens to accommodate their long CoT reasoning mechanisms. All inference processes are conducted on two NVIDIA A100-80G GPUs.

Model	Citation	Version
Claude-3-5-Sonnet	Anthropic (2024)	Claude-3-5-Sonnet-20241022
GPT-4o	OpenAI (2024)	gpt-4o-2024-08-06
GPT-4o-mini	OpenAI (2024)	gpt-4o-mini-2024-07-18
Qwen2-7B	Yang et al. (2024a)	Qwen/Qwen2-7B-Instruct
Qwen2.5-Coder-7B	Hui et al. (2024)	Qwen/Qwen2.5-Coder-7B-Instruct
Qwen2.5-Coder-32B	Hui et al. (2024)	Qwen/Qwen2.5-Coder-32B-Instruct
Qwen2.5-32B	Yang et al. (2024b)	Qwen/Qwen2.5-32B-Instruct
Qwen2.5-72B	Yang et al. (2024b)	Qwen/Qwen2.5-72B-Instruct
QwQ-32B-Preview	Team (2024b)	Qwen/QwQ-32B-Preview
Llama-3.1-8B	Meta (2024)	meta-llama/Llama-3.1-8B-Instruct
Llama-3.1-70B	Meta (2024)	meta-llama/Llama-3.1-70B-Instruct
Llama-3.3-70B	Meta (2024)	meta-llama/Llama-3.3-70B-Instruct
R1-Distill-Qwen-32B	DeepSeek-AI et al. (2025)	deepseek-ai/DeepSeek-R1-Distill-Qwen-32B
Codestral-22B	Team (2024a)	mistralai/Codestral-22B-v0.1
Mistral-Small-24B	Jiang et al. (2023)	mistralai/Mistral-Small-24B-Instruct-2501
Gemma-2-9B	Google (2024)	google/gemma-2-9b-it
Gemma-2-27B	Google (2024)	google/gemma-2-27b-it
StarChat2-15B	H4 (2024)	HuggingFaceH4/starchat2-15b-v0.1
StarCoder2-15B	Lozhkov et al. (2024)	bigcode/starcoder2-15b

Table 2: Model List.

C.2 CoT and Direct Output Prompts

The Chain-of-Thought Prompt in Task1

Task:
Generate a challenging test input for the algorithm problem:
{problem_description}

Instructions:
- Focus on edge cases or scenarios that maximize the failure probability in faulty solutions.
- Due to the output length limit, you should generate a small-scale test input that is complete and valid.
- Output the test input directly, not code to generate it.

Output format:

```
```plaintext
{test input}
```
```

Think step by step.

Figure 5: The Chain-of-Thought prompt used in Task1.

The Direct Output prompt in Task1

Task:
Generate a challenging test input for the algorithm problem:
{problem_description}

Instructions:
- Focus on edge cases or scenarios that maximize the failure probability in faulty solutions.
- Due to the output length limit, you should generate a small-scale test input that is complete and valid.
- Output the test input directly, not code to generate it.

Output format:

```plaintext
{test input}
```

Only output the test input, no explanations.

Figure 6: The Direct Output prompt used in Task1.

The Chain-of-Thought prompt in Task2

Task:
Generate a challenging test input that exposes the bug in the buggy code of the algorithm problem:
Algorithm Problem: {problem_description}
Buggy Code: {buggy_code}

Instructions:
- Focus on edge cases or scenarios that maximize the failure probability in faulty solutions.
- Due to the output length limit, you should generate a small-scale test input that is complete and valid.
- Output the test input directly, not code to generate it.

Output format:

```plaintext
{test input}
```

Think step by step.

Figure 7: The Chain-of-Thought prompt used in Task2.

The Direct Output prompt in Task2

Task:
Generate a challenging test input that exposes the bug in the buggy code of the algorithm problem:
Algorithm Problem: {problem_description}
Buggy Code: {buggy_code}

Instructions:
- Focus on edge cases or scenarios that maximize the failure probability in faulty solutions.
- Due to the output length limit, you should generate a small-scale test input that is complete and valid.
- Output the test input directly, not code to generate it.

Output format:

```plaintext
{test input}
```

Only output the test input, no explanations.

Figure 8: The Direct Output prompt used in Task2.